

**SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)**  
Department of Computer Science and Engineering

**ASSESSMENT TOOL 2– Concept Mapping**

|                         |                                                                                                                                                                                                                                                                                                                           |                      |                                     |
|-------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------|-------------------------------------|
| <b>Course Code:</b>     | CSA0313                                                                                                                                                                                                                                                                                                                   | <b>Course Title:</b> | Data Structures                     |
| <b>CO Assessed:</b>     | CO1 – Imitation (BL3, BL4)                                                                                                                                                                                                                                                                                                | <b>Assessment:</b>   | Assessment Tool 2 – Concept Mapping |
| <b>Weightage:</b>       | 10% of CIA                                                                                                                                                                                                                                                                                                                | <b>Total Marks:</b>  | 100                                 |
| <b>CO1 Statement:</b>   | Basics, Data, Data object, Data structure, Abstract Data Types (ADT), realization of ADT in 'C', Primitive and non-primitive data structure, Linear and non-linear data structure Arrays & Recursion, Performance analysis and Measurement: Time and space analysis of algorithms, Average, best and worst-case analysis. |                      |                                     |
| <b>Student Name:</b>    | S Susmitha                                                                                                                                                                                                                                                                                                                | <b>Reg. No.:</b>     | 192571068                           |
| <b>Section / Batch:</b> |                                                                                                                                                                                                                                                                                                                           | <b>Date:</b>         | 14.07.2026                          |

**Code of Conduct**

I S Susmitha (192571068) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: 

**Instructions**

- This test contains 2 concept mapping, Total: 100 Marks.
- Evaluation of Answer Quality -Checking whether the answer includes: Concept Identification , Linkages, Organization, Integration of Literature, and Clarity
- No DTP printouts or electronic devices permitted. They should provide materials in PDF format.

**Rubrics**

| Questions | Problem Understanding | Method Selection | Solution Process | Accuracy of Calculation | Interpretation of Results | Total Marks |
|-----------|-----------------------|------------------|------------------|-------------------------|---------------------------|-------------|
| Q1        |                       |                  |                  |                         |                           |             |
| Q2        |                       |                  |                  |                         |                           |             |

# SIMATS ENGINEERING

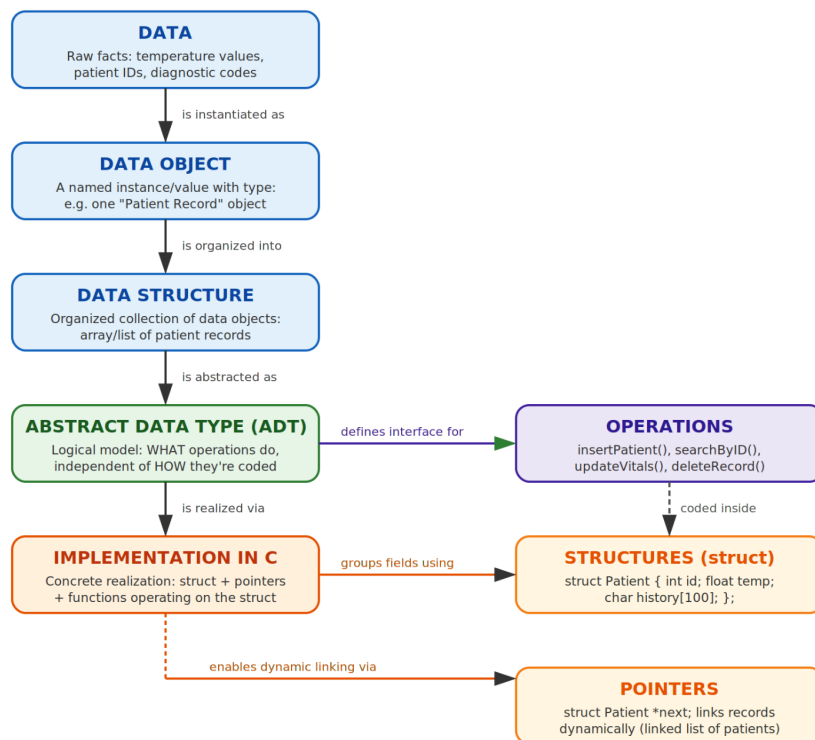
| Name       | Register Number | Course Code | Course          |
|------------|-----------------|-------------|-----------------|
| S Susmitha | 192571068       | CSA0313     | Data Structures |

## QUESTION 1

### CONCEPT MAP — DATA ORGANIZATION IN SMART HEALTHCARE SYSTEM

#### Q1 — Concept Map: Data → Data Object → Data Structure → ADT → C Implementation

Context: Smart Healthcare System (patient temperature, history, diagnostics)



## Explanation

A smart healthcare system processes a large amount of patient information such as **temperature readings, medical history and diagnostic details**. Proper organization of this data is essential for efficient storage, retrieval and processing.

### 1. Data

**Data** refers to raw facts and observations collected by the healthcare system. Temperature values, patient ID, medical history and diagnostic results are examples of data.

Data acts as the basic input for the healthcare system. Accurate data collection helps in reliable patient monitoring and processing.

## 2. Data Object

A **data object** is a meaningful collection of related data. In a healthcare system, the details of one patient are combined to form a **Patient Record**.

For example, a patient record contains the patient ID, temperature, medical history and diagnostic details.

Thus, related data is grouped into a meaningful object for easier processing.

## 3. Data Structure

A **data structure** systematically organizes multiple patient records.

An **array** can store a fixed number of patient records, whereas a **linked list** can dynamically store and connect patient records.

Proper data structures improve the storage, retrieval, insertion and deletion of healthcare information.

## 4. Abstract Data Type (ADT)

An **Abstract Data Type (ADT)** defines the logical behaviour and operations of patient record management without specifying the internal implementation.

The Patient ADT can perform operations such as:

- Insert patient
- Search patient
- Update patient vitals
- Delete patient record

Therefore, an ADT defines **what operations are performed**, while hiding the implementation details.

## 5. Implementation in C

The ADT is implemented in C using **structures, pointers and functions**.

A structure groups different patient attributes into a single record. Pointers support dynamic memory allocation and link patient records. Functions implement operations such as insertion, searching, updating and deletion.

Example:

```
struct Patient
{
    int id;
    float temperature;
    char history[100];
    struct Patient *next;
};
```

## Interpretation and Justification

The complete progression is:

**Data → Data Object → Data Structure → ADT → Implementation in C**

Raw healthcare data is combined to form patient data objects. These objects are systematically organized using data structures. ADT defines the required operations, and C implements these operations using structures, pointers and functions.

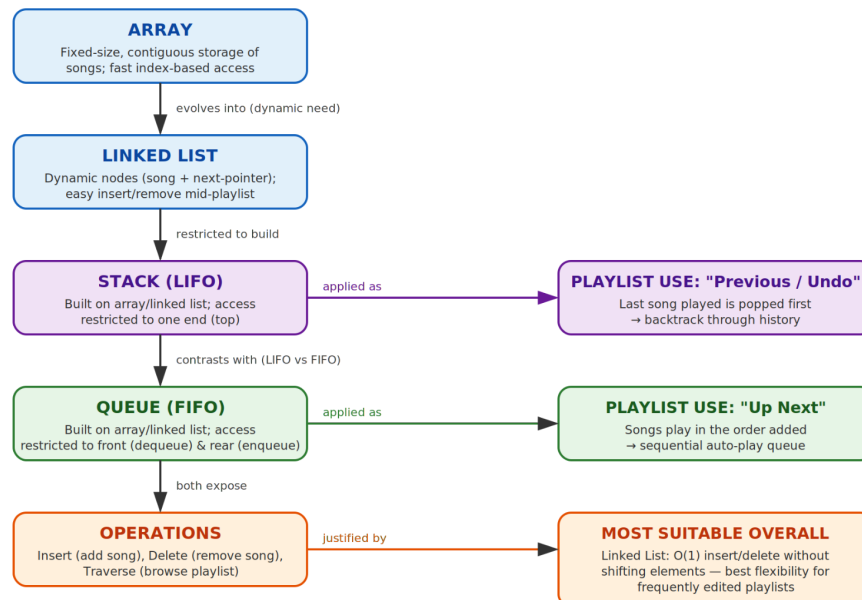
Hence, this hierarchical organization provides **efficient data storage, flexible record management, modularity and faster processing**, thereby improving the overall performance of the smart healthcare system.

## QUESTION 9

### CONCEPT MAP — DATA STRUCTURES IN PLAYLIST MANAGEMENT

#### Q9 — Concept Map: Array → Linked List → Stack → Queue → Operations

Context: Music Streaming Playlist Management



### Explanation

A music streaming application manages playlists in which songs are **added, removed and played sequentially**. Different data structures provide different operations and performance characteristics.

#### 1. Array

An **array** stores songs in contiguous memory locations. Each song can be accessed directly using its index.

Example:

Song A → Song B → Song C → Song D

Arrays provide **O(1) random access**, making them suitable for accessing a song by position.

However, insertion or deletion in the middle requires shifting elements and generally takes **O(n) time**. Therefore, arrays are less flexible for frequently changing playlists.

#### 2. Linked List

A **linked list** stores songs as dynamically connected nodes. Each node contains song information and a pointer to the next song.

Draw this small diagram:

[Song A | Next] → [Song B | Next] → [Song C | NULL]

A linked list supports dynamic playlist size and flexible modification.

Insertion and deletion can take **O(1) time when the required node or position is already known**. However, locating an arbitrary song may require **O(n) traversal**.

Unlike arrays, songs do not have to be shifted during link-based insertion and deletion.

### 3. Stack

A **stack** follows the **LIFO — Last In, First Out** principle.

It can be used to manage recent playback history or undo-like operations.

For example:

Song A → Song B → Song C

The most recently stored playback entry is handled first.

The main operations are:

- Push
- Pop
- Peek

Stack push and pop operations generally take **O(1) time**.

### 4. Queue

A **queue** follows the **FIFO — First In, First Out** principle.

It is suitable for the **Up Next** song list and sequential playback.

For example:

Song A → Song B → Song C

Song A is played first, followed by Song B and Song C.

The main operations are:

- Enqueue — Add a song
- Dequeue — Remove or play the first song

Queue operations generally take **O(1) time** when front and rear references are maintained.

### 5. Playlist Operations

**Insert** adds a new song to the playlist.

**Delete** removes an existing song from the playlist.

**Traverse** visits songs sequentially for displaying, searching or playing the playlist.

These operations directly affect the flexibility and performance of playlist management.

#### Performance Comparison

| Structure   | Application         | Performance            |
|-------------|---------------------|------------------------|
| Array       | Indexed song access | Access $O(1)$          |
| Linked List | Editable playlist   | Insert/Delete $O(1)^*$ |
| Stack       | Playback history    | Push/Pop $O(1)$        |
| Queue       | Up Next             | Enqueue/Dequeue $O(1)$ |

When the required linked-list node or position is known. Traversal to locate an arbitrary song may take  $O(n)$ .

#### Interpretation and Justification

Arrays provide fast random access but are less flexible for frequent insertion and deletion. Linked lists provide dynamic storage and flexible playlist modification. Stacks are suitable for playback history, while queues support sequential **Up Next** playback.

Therefore, the **linked list is the most suitable primary data structure for playlist management** because playlist size can dynamically increase or decrease and songs can be inserted or deleted without shifting multiple elements once the required position is located.

Hence, a **linked list provides flexibility for the main playlist, a stack manages playback history, and a queue manages sequential playback**, resulting in efficient and flexible playlist management.